화요일 코드 실행 안내

기준 커밋: 0a5c603ff9ed432a47e92958a745931dc7c1225f. START 외 명령의 전송 전 정지 대기와 요청 해제까지 DO0 유지 규칙을 반영했다.

빠른 함수·실패 실습은 PC 안의 교육 모형, 자기 코드 통합 실습은 가짜 PLC·로봇·PC의 세 프로그램으로 진행한다. 래더 문장을 가상 실행기가 스캔하고 로봇 신호를 연결한다. 가상장치 통과와 실물 성공은 별개다. 세 프로그램에는 제공 Modbus TCP 연결을 사용하며, 일반 TCP·HCR 직접 연결·MCP 강의는 후속 범위다.

1. 준비와 폴더

Windows PowerShell에서 학생은 학생코드, 강사는 강사코드 폴더를 연다. 다음 명령은 모두 그 폴더 안에서 실행한다. picknplace_full 하위 폴더 안으로 들어가 실행하지 않는다.

```
python --version
python -m pip install -r requirements.txt
python -B -X utf8 -m picknplace_full --target model
```

Python 3.11 이상이 필요하다. 제작 검증 환경은 Python 3.13.5, pymodbus 3.16.1, pyserial 3.5다. -B는 캐시 파일 생성을 막고 -X utf8은 한글 출력 인코딩을 맞춘다. -m은 폴더에 있는 프로그램을 모듈로 실행한다. --target model은 실제 COM 포트를 찾지 않고 모형만 선택한다. 원본 앱의 생략 기본값은 auto이므로 수업 명령에서 --target model을 빼지 않는다.

브라우저에서 http://127.0.0.1:8877을 연다. 화면의 대상이 model인지 확인한다. 처음에는 모형 PLC의 초기 그리퍼 열림 타이머가 진행되므로 기본 배속에서 약 3초 후 버튼을 시험한다. 이 시간은 준비 타이머이며 수업 시간 배분과는 무관하다. 모형은 실제 로봇을 움직이지 않는다.

같은 모델 앱의 안전한 실행 입구도 제공한다.

```
python -B -X utf8 lesson_ui.py
```

이 실행 입구에는 실물 대상을 고르는 옵션이 없다. 두 명령을 동시에 실행하지 않는다. 포트 사용 중 오류가 나면 먼저 실행한 자기 터미널에서 Ctrl+C로 끝낸다. 8878은 아래 가짜 PLC 화면이 쓰므로 PC 앱의 대체 포트로 쓰지 않는다. 프로그램 종료는 PC 작업만 끝낸다. 실제 장치의 정지 명령과 같은 뜻이 아니다.

2. 제공 부분과 학생 수정 부분

파일	역할	수업 중 작업
picknplace_full/transport.py	실제 Modbus 또는 모형에 읽기·쓰기를 전달	제공, 수정하지 않음
picknplace_full/link.py	통신 작업 한 개로 요청을 순서대로 처리하고 새 관측을 알림	제공, 호출 위치 확인
picknplace_full/status.py	상태 숫자를 로봇·그리퍼·PLC 판정으로 해석	제공, 실제 관측과 대조

picknplace_full/sequence.py	PC가 다음 동작을 결정하고 버튼 허용을 판단	A/B/C/D 수정
picknplace_full/app.py, web/	UI와 제어부를 연결	제공
lesson_scenarios.py	교육 모형의 고장·대기·회복 재현	실행하고 로그 비교
check_student.py	해당 폴더의 실제 학생 함수를 호출하는 동작 검사	각 수정 뒤 실행
fake_plc/, fake_robot/	독립 PLC·로봇 실행기와 관측 화면	제공, 세 프로그램 통합에 사용
ladder/ PICKNPLACE_FULL.xgwx	가짜 PLC가 실행할 현행 래더	파일명·RUN·접점·타이머 확인
robot/ PICKNPLACE_FULL_main.js	파일 모드에서 고를 제공 로봇 스크립트	선택·다시 선택, 실행 로그 확인

학생 시작본의 A와 B는 NotImplementedError로 표시된다. C는 정상 상태 대기를 제공하고 시간 초과 처리만 미구현이다. D는 TODO 주석이며 초기에는 원본의 버튼 정책을 유지한다. 시작본 UI의 관측·개별 이동·그리퍼·즉시 정지·재개는 제공 부분으로 실행된다. 자동 시작은 A/B를 완성해야 정상 실행된다.

3. 단계별 검사

```
python -B -X utf8 check_student.py --stage A
python -B -X utf8 check_student.py --stage B --part normal
python -B -X utf8 check_student.py --stage B --part stop
python -B -X utf8 check_student.py --stage C
python -B -X utf8 check_student.py --stage D
python -B -X utf8 check_student.py --stage all
```

단계	검사 수	배포 시작본의 예상 결과
A	5	판정·START 예외·전송 전 대기 모두 미구현 실패
B normal	1	미구현 실패
B stop	3	미구현 실패
C	2	새 관측 대기 통과, 시간 초과 미구현 실패
D	1	복구 전 개별 버튼 보류 정책이 없어 실패
all	12	1 통과, 11 실패

이 실패는 설치 오류가 아니라 학생 작업 자리의 표시다. 강사 완성본은 12개 통과가 정상이다. B 검사는 다른 미완성 과제의 영향을 받지 않도록 A/C를 시험 대역으로 바꾸고 학생 _cycle을 직접 실행한다. A는 START 외 명령의 전송 전 정지 대기도 검사하며 C 미완성의 영향을 없애기 위해 대기 부품을 시험 대역으로 제공한다. 검사 통과는 특정 조건의 코드 동작 확인이며 실물 성공 확인이 아니다.

4. 고장과 복구 모형

```
python -B -X utf8 lesson_scenarios.py --case observe
python -B -X utf8 lesson_scenarios.py --case normal
python -B -X utf8 lesson_scenarios.py --case stop
python -B -X utf8 lesson_scenarios.py --case pause
python -B -X utf8 lesson_scenarios.py --case delay
python -B -X utf8 lesson_scenarios.py --case read_fail
python -B -X utf8 lesson_scenarios.py --case read_timeout
python -B -X utf8 lesson_scenarios.py --case write_reply_lost
python -B -X utf8 lesson_scenarios.py --case final_open_estop
```

사례	필요한 학생 구현	예상 결과와 확인할 근거
observe	없음	상태 raw=2, mailbox=0. 2222 확인 요청을 보내 접수 결과와 새 관측 시각 비교
normal	A/B 정상	result: 완료, writes: [1111,9991,9992]. 그리퍼 자동 동작은 PLC가 맡음
stop	A/B 정지	이동1 중 요청, 닫힘 확인 뒤 9994, 결과 정지. 9992 없음
pause	없음	writes: [9991,9995,9993], 모형 정지·재개 확인
delay	없음	접수 대기 중 heartbeat_ticks 증가. 수치는 PC에 따라 다름
read_fail	없음	읽기 실패 중 마지막 관측 시각 유지, 회복 뒤 새 관측. 회복 자체가 전체 완료는 아님
read_timeout	A/B/C	이동1 대기 시간 초과, writes: [1111,9991], 다음 이동2 없음. 통신 회복 뒤에도 자동 재시작 없음
write_reply_lost	없음	명령 결과 미확인, 모형은 이동 시작, write_count: 1. 자동 재전송 없음
final_open_estop	A/B 정상	마지막 열림 중 정지 후 결과 완료, resume_sent: false, robot_abnormal_remains: true

각 명령은 새 모형을 만들어 한 사례를 끝낸다. 다른 CLI 실행에서 앞 작업이 이어지지 않는다. 이미 켜 둔 세 프로그램 가상장치에도 이 고장이 주입되지 않는다. --speed 5는 검증 편의를 위한 모형 5배속이며 실제 장치의 속도를 정하지 않는다. pause의 내부 이동 정지 확인은 모형 구현의 결과이며 실제 HCR Thread의 검증이 아니다.

5. 같은 UI 실행에서 복구 정책 비교

```
python -B -X utf8 lesson_ui.py --fault write_reply_lost
```

1. 준비 후 시작을 누른다. A/B가 완성되어 있어야 한다.
2. 첫 9991은 모형에 전달되지만 응답만 한 번 유실된다. 화면·로그는 미확인 중단을 표시하고 auto_maybe_on이 참이 된다.

3. 모형 이동과 자동 그리퍼 닫힘이 끝난 후 개별 이동1·이동2·그리퍼의 허용 상태를 비교한다. D 미완성은 원본처럼 허용, D 완성은 보류다. 이 비교 중 개별 동작을 실제로 눌러 혼선을 추가하지 않는다.
4. 같은 실행에서 자동 운전 해제(9994)를 누른다. 기존 정지 버튼의 문구가 복구 필요 상태에 따라 이 이름으로 바뀐 것이다. 접수가 확인되면 복구 추정이 해제된다. 정상 상태이면 개별 버튼을 다시 허용한다.
5. 그리퍼가 닫혔다면 개별 그리퍼 열기를 누르고 열림 관측 후 다음 시작을 시험한다. 주입한 고장은 첫 9991에 한 번만 일어난다.

소스 수정은 실행 중 앱에 자동 반영되지 않는다. 수정 전 조건은 화면·로그로 기록하고 종료한 뒤 수정·재실행해 동일 고장 조건을 다시 만든다. **수정 후 실행 안에서** 9994 전후의 변화를 확인한다. 프로그램을 껐다 켜 초기 상태가 된 것은 같은 작업을 복구한 증거가 아니다.

6. 결과가 다를 때

현상	확인과 대응
No module named …	코드 폴더 루트인지, 같은 python으로 requirements를 설치했는지 확인
NotImplementedError 과제 A/B/C	안내된 학생 작업 자리인지 확인. 해당 단계 검사부터 실행
명령이 거부됨	최신 상태, 우편함, PLC 판정, 초기화 타이머, 자동 운전 조건을 대조. 즉시 재전송하지 않음
쓰기 결과 미확인	장치가 이미 받았을 수 있음. 마지막 관측 시각·새 상태·로그를 확인
고장 사례에서 UI 변화가 안 보임	CLI는 별도 모형이다. UI용 고장은 lesson_ui.py --fault write_reply_lost로 재현
D 수정이 반영되지 않음	실행 중 앱 종료 후 수정한 폴더에서 재실행. 다른 코드 폴더를 열지 않았는지 확인

7. 실물 적용 경계

실물 적용은 강사가 장치별 설정과 현행 로봇 스크립트를 확인한 뒤 별도 진행한다. 이 패키지는 고정 커밋의 문서·코드·모형 근거를 제공한다. 로봇 DO0은 DI0·DI1 요청이 꺼질 때까지 유지하는 최신 규칙을 반영했다. 티칭 좌표·장치 설정·실물 응답을 확인하기 전에 스크립트나 래더를 임의로 업로드하지 않는다. 현재 통신 구현의 Modbus 주소 0/1은 PLC D01000/D01001과 매핑된다는 설정 전제이며, 실물 설정과 반드시 대조한다. 주소 숫자만 보고 물리 장치의 메모리 주소를 확정하지 않는다. 제공 9995 버튼은 통신으로 보내는 기능 정지 요청이며 안전 비상정지 수단이 아니다.

수업의 현행 래더 파일은 ladder/PICKNPLACE_FULL.xgwx와 대응 .ld.txt다. 강사 폴더에만 남긴 e2e_pickplace와 tests/samples는 구계약의 회귀 시험 자료이며 현행 수업용 업로드 파일이 아니다. 학생 묶음에서는 제외했다. 로봇 폴더의 제공 스냅샷은 가짜 로봇 파일 모드에서 선택할 수 있다. 실제 장비 적용 전에는 robot/수업_확인사항.md의 미확인 조건과 현장 티칭 값을 대조한다.

8. 세 프로그램으로 자기 코드 통합

상세 순서와 실패 대응은 가상장치실습안내를 따른다. 학생은 같은 학생코드 폴더에서 터미널 세 개를 열고 아래를 하나씩 실행한다. 각 줄의 프로그램은 켜 둔다.

```
python -B -X utf8 -m fake_plc ladder/PICKNPLACE_FULL.xgwx --modbus-port 5020 --wiring-port 5021 --http-port 8878
```

```
python -B -X utf8 -m fake_robot --plc-host 127.0.0.1 --wiring-port 5021 --http-port 8879
```

```
python -B -X utf8 -m picknplace_full --target tcp:127.0.0.1:5020 --http-port 8877
```

화면은 PLC http://127.0.0.1:8878, 로봇 http://127.0.0.1:8879, PC http://127.0.0.1:8877이다. 이 PC 앱이 같은 폴더의 자기 picknplace_full/sequence.py를 사용한다. 가짜 PLC RUN 직후 약 3초 동안은 상태 0002와 시작 버튼 활성만으로 준비를 확정하지 않는다. M00418 열기 진행이 끝난 뒤 조작한다.

내장 로봇에서 연결을 확인하고, 로봇 화면에서 같은 폴더의 robot/PICKNPLACE_FULL_main.js를 선택해 파일 실행을 확인한다. 파일 수정 후 다시 선택해야 새 내용을 읽는다. 파일 모드에서는 브라우저 탭을 열어 둔다. Node는 학생 실습의 선행 조건이 아니다.